



REALM FORGE

AI-Generated Team Conquest RPG

Mistral Worldwide Hackathon — Build Plan & Technical Design Document

1. Project Overview

Realm Forge is a browser-based, multiplayer team conquest RPG where the entire game world — its map, factions, quests, enemy behavior, and lore — is procedurally generated by Mistral AI at the start of every session. No two games are the same. Teams of 2–4 players collaborate to conquer regions, level up their characters, and face a dynamically adaptive final boss whose strategy evolves based on how the team plays.

The game targets the intersection of Supercell-style accessible team gameplay with the power of large language models as a real-time dungeon master. Players interact via text commands, map clicks, and optional voice input, receiving narrated, voiced responses from the AI after every action.

1.1 Concept Pillars

- Every session is a unique AI-generated world built from a player's text prompt
- Teams work together — actions are shared, strategy is collaborative
- Mistral is the dungeon master — it narrates, adapts, and remembers player decisions
- Voice-first optional layer — ElevenLabs voices every character and narration
- Retro pixel aesthetic — fast to build, instantly recognizable, Supercell-inspired charm

1.2 Target Prize Categories

Prize	Sponsor	Why We Qualify
Best Video Game Project	Supercell	Full team conquest RPG with Supercell-style mechanics and AI-powered gameplay loop
Best Use of ElevenLabs	ElevenLabs	Every character, faction, and DM narration is voiced using ElevenLabs API
Best Architectural Modification	Tilde Research	Novel generative pipeline: world JSON → rendered game → adaptive AI DM
The Hackathon's Next Unicorn	Giant VC	AI-generated infinite games is a genuine product category with commercial potential
Best Vibe Usage	Mistral	Mistral is the core engine — world gen, game logic, and DM narration all powered by Mistral models

2. Game Design

2.1 Core Game Loop

The session flow is designed to be completable in 20–40 minutes — short enough to demo at a hackathon, deep enough to feel satisfying.

```

Team assembles (2-4 players)
    ↓
One player describes the world: e.g. 'volcanic island, pirate clans, betrayal theme'
    ↓
Mistral Large 3 generates: map, factions, quests, boss mechanics, lore
    ↓
Team explores map — click regions, type commands, or speak via push-to-talk
    ↓
Ministral 8B processes actions → DM narrates outcome → ElevenLabs voices it
    ↓
Team earns XP → levels up → unlocks new abilities
    ↓
Final boss fight — difficulty and strategy adapted to team playstyle
    ↓
Victory screen + shareable world summary card

```

2.2 World Structure

Each generated world consists of the following elements, all produced by Mistral Large 3 as a structured JSON object:

Element	Description	Example
World Name	AI-generated title for the realm	"The Ashen Archipelago"
Regions (4–6)	Named map areas, each with a faction and difficulty tier	"Cinder Bay — Pirates — Medium"
Factions (2–3)	Enemy groups with distinct behaviors and personalities	"The Ironclad Brotherhood"
Quests (3–4)	Optional side quests that affect the final boss	"Recover the stolen compass"
Boss	Final region with adaptive mechanics and unique lore	"Admiral Vex, the Betrayer"
World Lore	2–3 sentence backstory injected into DM narration	"This island was once peaceful..."

2.3 Player Actions

Players can interact with the game in three ways — all three resolve to the same game logic layer:

- **Click on a map region** → opens an action panel (attack, scout, fortify, negotiate)
- **Type a natural language command** → 'Send the warriors north and scout first'
- **Push-to-talk voice** → Voxtral Realtime transcribes → same pipeline as text

2.4 Team Mechanics

- All players share one game state — actions are visible to the whole team in real time
- Each player has a role (Warrior, Scout, Mage, Diplomat) with unique ability bonuses
- Team XP is shared — leveling up unlocks one new ability for the whole team
- The DM tracks each player's decisions — betrayal, cowardice, or bravery affects the story

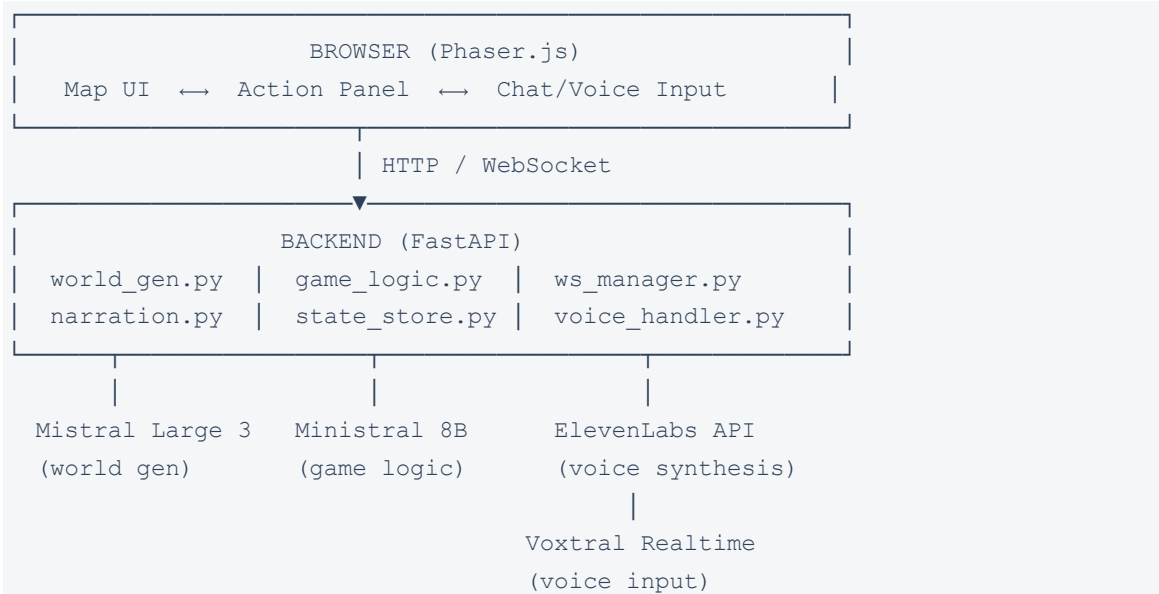
2.5 MVP Scope vs Nice-to-Have

Feature	Priority	Notes
World generation from prompt	Must Have	Core of the entire experience

Feature	Priority	Notes
Map rendering with clickable regions	Must Have	Phaser.js, static sprites fine
Text command input	Must Have	Natural language → game action
DM narration (text)	Must Have	Mistral 8B response after every action
Multiplayer sync (2–4 players)	Must Have	Shared state via WebSocket
ElevenLabs voice narration	Must Have	Needed for ElevenLabs prize
Character role selection	Must Have	Simple — 4 roles, different stat bonuses
Final boss fight	Must Have	Adaptive difficulty based on team performance
Voice input (Voxtral)	Nice to Have	Push-to-talk, add in Phase 5
Shareable world summary card	Nice to Have	Great for demo, add if time allows
Animations beyond basic	Cut	Static sprites are fine for hackathon
Mobile optimization	Cut	Desktop browser only
Persistent user accounts	Cut	Single session, no auth needed

3. Technical Architecture

3.1 System Overview



3.2 Folder Structure

```
realm-forge/
├── frontend/
│   ├── scenes/
│   │   ├── WorldScene.js    ← main map, region rendering
│   │   ├── BattleScene.js   ← boss fight sequence
│   │   └── UIScene.js       ← overlaid HUD, chat, voice button
│   ├── assets/
│   │   ├── sprites/        ← pixel art from itch.io free packs
│   │   └── audio/          ← ambient sound, ElevenLabs cached audio
│   └── api/
│       ├── worldApi.js      ← world gen call
│       ├── actionApi.js     ← player action call
│       └── wsClient.js       ← WebSocket game state sync
├── backend/
│   ├── main.py              ← FastAPI app, routes
│   ├── world_gen.py         ← Mistral Large 3 world generation
│   ├── game_logic.py        ← Ministral 8B action processing
│   ├── narration.py         ← ElevenLabs integration
│   ├── voice_handler.py     ← Voxtral Realtime pipeline
│   ├── state_store.py       ← in-memory game state manager
│   └── ws_manager.py        ← WebSocket broadcast manager
├── prompts/
│   ├── world_gen_system.txt ← Mistral Large 3 system prompt
│   ├── dm_system.txt        ← DM narration system prompt
│   └── boss_system.txt       ← Boss adaptive AI prompt
└── docker-compose.yml       ← spin up backend + Redis for state
```

3.3 Model Usage

Model	Task	Why This Model
Mistral Large 3	World generation from player prompt	Needs deep reasoning to build coherent lore, factions, and interconnected quests. Quality matters more than speed here — called once per session.
Ministral 8B	Real-time game logic and DM narration	Fast inference needed — called after every player action. 8B hits the sweet spot of quality and latency.
Voxtral Realtime	Voice input transcription (push-to-talk)	Native Mistral voice model. Processes player speech and returns intent as text into the same action pipeline.

Model	Task	Why This Model
ElevenLabs	Voice synthesis for DM narration and characters	Unique voices per faction leader and DM narrator. Called after every Ministral 8B response.

3.4 Data Flow — Player Action

1. Player clicks region OR types command OR speaks (push-to-talk)
2. If voice → Voxtral Realtime → transcribed text
3. Action sent to FastAPI /action endpoint
4. game_logic.py builds prompt: [game state] + [player action] + [dm_system.txt]
5. Ministral 8B returns: { outcome, state_changes, narration_text, tone }
6. state_store.py updates game state
7. ws_manager.py broadcasts new state to all connected players
8. narration.py sends narration_text to ElevenLabs → returns audio URL
9. Frontend plays audio + updates map UI

4. Core Mistral Prompts

4.1 World Generation Prompt (Mistral Large 3)

This is the most critical prompt in the entire project. The quality of every downstream game element depends on it. The system prompt instructs Mistral to act as a world-builder and return a strict JSON schema. The user message is the player's world description.

System Prompt — world_gen_system.txt

```
You are a world-builder AI for a retro fantasy conquest game.
Your job is to generate a complete, rich game world from a player's
description.

RULES:
- Return ONLY valid JSON. No markdown, no explanation, no preamble.
- The world must feel internally consistent – factions should have history with
  each other.
- Every region must have a unique strategic value (resource, information,
  military advantage).
- The boss must feel like a natural culmination of the world's conflict.
- Side quests must affect the boss fight if completed (reward player
  exploration).
- Tone and theme must match the player's prompt – dark, playful, epic, etc.
```

OUTPUT SCHEMA:

```
{
  "world_name": "string",
  "lore": "string (2-3 sentences of world backstory)",
  "theme": "string (e.g. pirate betrayal, dark magic, merchant war)",
  "factions": [
    {
      "id": "string",
      "name": "string",
      "description": "string",
      "personality": "aggressive | cunning | defensive | diplomatic",
      "voice_tone": "string (for ElevenLabs - e.g. gruff, whispering, jovial)"
    }
  ],
  "regions": [
    {
      "id": "string",
      "name": "string",
      "description": "string",
      "faction_id": "string",
      "difficulty": "easy | medium | hard",
      "strategic_value": "string",
      "position": { "x": 0-100, "y": 0-100 }
    }
  ],
  "quests": [
    {
      "id": "string",
      "title": "string",
      "description": "string",
      "region_id": "string (where to find it)",
      "boss_impact": "string (how completing this weakens/changes the boss)"
    }
  ],
  "boss": {
    "name": "string",
    "description": "string",
    "faction_id": "string",
    "base_difficulty": 1-10,
    "adaptive_behavior": "string (how boss adapts to team's playstyle)",
    "weakness": "string (revealed only if team completed specific quests)",
    "voice_tone": "string"
  }
}
```

Example User Message

```
volcanic island, pirate clans, theme of betrayal, medium difficulty
```

4.2 DM Narration Prompt (Minstral 8B)

Called after every player action. The system prompt establishes the DM's voice and rules. The full current game state is injected as context on every call — this is stateless on the model side, stateful on our server side.

System Prompt — dm_system.txt

```
You are the Dungeon Master for a real-time multiplayer conquest game.
You narrate outcomes, control enemy behavior, and keep the story moving.

RULES:
- Keep narration to 2-4 sentences. Vivid but concise — this will be read aloud.
- Always reference the world's theme and lore naturally — don't just describe mechanics.
- Track and remember player decisions. Reward clever play. Punish recklessness.
- Enemy behavior must match their faction personality (aggressive/cunning/defensive).
- If a quest was completed, reference its impact on the current situation.
- End every narration with a hook — a hint, a threat, or a temptation.

Return JSON only:
{
  "narration": "string (2-4 sentences, will be voiced by ElevenLabs)",
  "outcome": "success | partial | failure",
  "state_changes": {
    "xp_gained": number,
    "region_status": "conquered | contested | lost | unchanged",
    "quest_triggered": "quest_id or null",
    "boss_modifier": "string or null (if player action affects boss)"
  },
  "tone": "epic | tense | mysterious | humorous | ominous"
}
```

4.3 Boss Adaptive Prompt

Called at the start of the boss fight. Mistral Large 3 analyzes the team's full session history and generates a customized boss encounter — adjusting difficulty, tactics, and dialogue to reflect how the team played.


```
Given this team's session history: [HISTORY]
And the boss definition: [BOSS_JSON]
And completed quests: [COMPLETED_QUESTS]

Generate a final boss encounter tailored to this team:
{
  "opening_monologue": "string (boss taunts the team based on their
decisions)",
  "adapted_difficulty": 1-10,
  "special_mechanics": ["string"],
  "weakness_revealed": boolean,
  "victory_narration": "string",
  "defeat_narration": "string"
}
```

5. Build Phases & Timeline

The build is structured across 3 days. Each phase has a clear deliverable — if a phase runs long, the next phase gets trimmed, not the core loop.

Phase 1 — World Engine (Day 1, Morning)

Goal: Mistral Large 3 reliably returns a well-structured world JSON from a plain text prompt.

- Set up FastAPI project structure and install dependencies
- Write `world_gen_system.txt` prompt and iterate until output is consistently clean JSON
- Build `/generate-world` endpoint — takes player prompt, returns world JSON
- Write a simple test script that calls the endpoint with 5 different prompts and checks JSON validity

Exit criteria: 5/5 prompts return valid, coherent world JSON with all required fields.

Phase 2 — Map Rendering (Day 1, Afternoon)

Goal: A playable Phaser.js map renders from world JSON. Players can click regions.

- Set up Phaser.js project, install free pixel art asset pack from itch.io
- Write `WorldScene.js` — reads world JSON, places region nodes at x/y positions
- Add click handlers on regions — opens action panel (hardcoded actions for now)
- Add basic HUD — team XP bar, player role display, current region info
- Wire up static dummy game state to test the UI — no AI needed yet

Exit criteria: A hardcoded world JSON renders a clickable map with region panels.

Phase 3 — Game Loop + DM Narration (Day 2, Morning)

Goal: Player actions flow through Ministral 8B and DM narration is voiced by ElevenLabs.

- Write `game_logic.py` — takes player action + game state, calls Ministral 8B
- Write `narration.py` — takes narration text, calls ElevenLabs, returns audio URL
- Wire text command input box to action pipeline
- Wire map click actions to same pipeline
- Frontend plays ElevenLabs audio after each action
- Test full loop: click region → action panel → text command → DM response → voice

Exit criteria: A complete action loop works end-to-end with voice narration.

Phase 4 — Multiplayer Sync (Day 2, Afternoon)

Goal: 2–4 players share a live game state. All players see each other's actions.

- Write `ws_manager.py` — WebSocket manager, room-based (one room per game session)
- Write `state_store.py` — in-memory dict keyed by session ID, stores full game state
- All `/action` calls update state and broadcast to room via WebSocket
- Frontend `wsClient.js` receives state updates and re-renders map
- Add player join flow — enter session code, choose role, join game

Exit criteria: Two browser tabs share one game and see each other's actions live.

Phase 5 — Polish + Voice Input (Day 3)

Goal: Demo-ready build. Voice input added, rough edges fixed, demo script practiced.

- Add Voxtral push-to-talk button — hold to speak, release to send
- Add final boss fight scene — `BattleScene.js`, boss adaptive prompt, boss voice
- Add victory screen with shareable world summary (world name, key decisions, team score)
- UI polish — make it look retro-game good, not prototype bad
- Practice demo script: world prompt → world appears → 2 actions → boss fight → win
- Prepare 60-second elevator pitch for judges

Exit criteria: A complete, demo-able game session from world prompt to victory screen.

6. Full Tech Stack

Layer	Technology	Purpose
Game Engine	Phaser.js 3	2D browser game rendering, scene management, input handling
Frontend Framework	Vanilla JS + Vite	Fast dev setup, no framework overhead needed for a hackathon
Backend	FastAPI (Python)	Async API, easy WebSocket support, fast to write
AI — World Gen	Mistral Large 3	Complex world generation requiring deep reasoning
AI — Game Logic	Ministral 8B	Fast real-time DM narration and game state updates
AI — Voice Input	Voxtral Realtime	Speech-to-intent for push-to-talk commands
Voice Synthesis	ElevenLabs API	Character and DM narration voices
Multiplayer	FastAPI WebSockets	Real-time game state sync — no external service needed
State Management	In-memory dict (Python)	Simple, fast, no DB needed for a hackathon session
Pixel Art Assets	itch.io free packs	Fantasy RPG sprites — Kenney.nl has excellent free packs
Deployment	Google Cloud Run / Render	Fast free-tier deploy for hackathon demo

7. Risks & Mitigations

Risk	Likelihood	Mitigation
Mistral JSON output malformed	Medium	Add JSON validation + retry logic on world_gen.py. Test prompt extensively on Day 1.
ElevenLabs latency too high	Medium	Cache common DM phrases. Pre-generate faction leader voice intros at world gen time.
Voice input STT errors (Voxtral)	Medium	Push-to-talk prevents overlap. Add fallback text input always visible. Voice is Nice-to-Have.
Multiplayer state conflicts	Low	Action queue — process one action at a time. Last-write wins for hackathon.

Risk	Likelihood	Mitigation
Team can't finish in time	Medium	Phases 1–3 give a solo-playable demo. Multiplayer and voice are additive layers.
Pixel art assets look bad	Low	Kenney.nl RPG pack is production quality and free. Allocate 30 min to asset selection Day 1.

8. Demo Script

The demo should take 60–90 seconds and be practiced at least 5 times before judging. Judges see dozens of projects — you need to hit the wow moment within the first 10 seconds.

The Pitch (10 seconds)

"What if every game session was a unique world designed by AI, narrated by AI, and voiced in real time? That's Realm Forge."

The Demo Flow (60 seconds)

1. Type the world prompt live: "volcanic island, rival pirate clans, betrayal theme"
2. World appears on screen in 3–5 seconds — map, faction names, lore text
3. Click a region → ElevenLabs DM voice narrates the outcome with the right tone
4. Type a command: "negotiate with the Ironclad Brotherhood" — DM responds in character
5. Show second browser tab — a teammate's action updates your map in real time
6. Trigger boss fight — boss taunts the team based on their decisions, voiced by ElevenLabs

The Close (10 seconds)

"Every prompt, every team, every session — a completely different world. Infinite replayability, powered entirely by Mistral."

9. Useful Resources

Resource	URL / Notes
Mistral API Docs	docs.mistral.ai — model list, API reference, rate limits
Ministral Models on HF	huggingface.co/mistralai — for fine-tuning track reference
ElevenLabs API	elevenlabs.io/docs — voice synthesis, voice cloning, latency tips
Voxtral Realtime	Check Mistral hackathon Discord for Voxtral access and docs
Phaser.js	phaser.io/learn — official tutorials, very good docs
Kenney.nl Pixel Art	kenney.nl/assets — free CC0 RPG/fantasy sprite packs
FastAPI WebSockets	fastapi.tiangolo.com/advanced/websockets/ — official guide
itch.io Free Assets	itch.io/game-assets/free/tag-pixel-art — community packs

Realm Forge — Mistral Worldwide Hackathon 2026
Built with Mistral Large 3 · Ministral 8B · Voxtral · ElevenLabs · Phaser.js